

RA1 - AEE Consumo de la API de la NASA y Modificación de Llamadas en Java

Resultados de Aprendizaje y Conceptos básicos

Resultado de Aprendizaje: Aplica tecnologías de desarrollo para dispositivos móviles evaluando sus características y capacidades.

Criterios evaluación:

- a) Se han analizado las limitaciones que plantea la ejecución de aplicaciones en los dispositivos móviles.
- b) Se han identificado las tecnologías de desarrollo de aplicaciones para dispositivos móviles.
- c) Se han instalado, configurado y utilizado entornos de trabajo para el desarrollo de aplicaciones para dispositivos móviles.
- d) Se han identificado configuraciones que clasifican los dispositivos móviles en base a sus características.
- e) Se han descrito perfiles que establecen la relación entre el dispositivo y la aplicación.
- f) Se ha analizado la estructura de aplicaciones existentes para dispositivos móviles identificando las clases utilizadas.
- g) Se han realizado modificaciones sobre aplicaciones existentes.**
- h) Se han utilizado emuladores para comprobar el funcionamiento de las aplicaciones.

Contenidos básicos:

Análisis de tecnologías para aplicaciones en dispositivos móviles:

- Limitaciones que plantea la ejecución de aplicaciones en los dispositivos móviles, desconexión, seguridad, memoria, consumo batería, almacenamiento.
- Entornos integrados de trabajo.
- Módulos para el desarrollo de aplicaciones móviles.
- Emuladores.
- Configuraciones. Tipos y características. Dispositivos soportados.
- Perfiles. Características. Arquitectura y requerimientos. Dispositivos soportados.
- Ciclo de vida de una aplicación, descubrimiento, instalación, ejecución, actualización y borrado.
- Modificación de aplicaciones existentes.
- Utilización del entorno de ejecución del administrador de aplicaciones.

Contenidos Avanzados:

- Flutter
- Apache Cordova (Antes Phonegap)

Competencias Personales

Resumen

- Pendiente

Anexos

- **Glosario de Términos**
- **Ejercicios Prácticos y Proyectos**
- **Casos de Estudio y Ejemplos Reales**

Glosario

RA: Resultado de aprendizaje

CEV: Criterio de evaluación
AF: Actividades formativas
AEE: Actividades específicas evaluables
AEER: Actividades específicas evaluables de recuperación
AERF: Actividades específicas de refuerzo
AEEA: Actividades específicas evaluables de ampliación
AEFA: Actividades específicas formativas de ampliación
AC: Actividades complementarias

1. **Desarrollo código 70%:**
 - El alumno deberá desarrollar el objetivo y requisitos del anexo.
2. **Documentación 20%:**
 - Todo el código deberá ir documentado.
3. **Entrega 0%:**
 - **Apellidos_Nombre_.zip 0%** Un zip o rar con el proyecto completo.
 - **Link a proyecto GIT del Alumno + readme con la información 0%**
4. **Competencias personales 10% (1 punto)**
 - **Iniciativa y Resolución de Conflictos**

Fecha de Entrega: Definir en Classroom

Sin comentarios adicionales

Pasos de la práctica:

Introducción:

En esta práctica, aprenderás a consumir la API pública de la NASA para obtener la imagen del día y a modificar una clase Java para hacer que la llamada a la API incluya una fecha aleatoria. Además, tendrás un reto adicional: extraer la información de la imagen y mostrarla en TextViews en tu Activity.

¿Qué es la API de la NASA para la imagen del día?

La API de la NASA para la "Astronomy Picture of the Day" (APOD) permite acceder a la imagen astronómica destacada del día, junto con una breve descripción y otros detalles. Esta API es gratuita, pero requiere que te registres para obtener una clave de acceso (API Key).

Objetivos:

1. **Objetivo 1:** Configurar tu propia cuenta y clave de API en la plataforma de la NASA para consumir la API.
2. **Objetivo 2:** Modificar el código base proporcionado para que la clase realice solicitudes con una fecha aleatoria automáticamente.
3. **Bonus 1:** Extraer y mostrar la información de la imagen (título, fecha y explicación) en TextViews en la Activity.
4. **Bonus 2:** Explorar y usar otro método de la API de la NASA, como por ejemplo, obtener datos de imágenes de Marte (Mars Rover Photos) o información de asteroides (Asteroids – NeoWs).

Requisitos previos:

- Tener conocimientos básicos de Java.
- Familiaridad con la estructura de proyectos en Java y Android.
- Entender cómo funcionan las solicitudes HTTP y el manejo de respuestas JSON.

Pasos de la práctica:

Objetivo 1: Configuración de la API

1. **Registro en la API de la NASA:**
 - Ve a <https://api.nasa.gov/> y regístrate para obtener una clave de API gratuita.
 - Una vez que tengas tu clave de API, guárdala, ya que la necesitarás para la práctica.
2. **Incorporar la clave en el código base:**
 - Reemplaza la clave de API en el código base que te proporcioné con tu propia clave obtenida en el paso anterior.

Objetivo 2: Modificación del código para generar una fecha aleatoria

1. **Código base:** Se te proporcionará un archivo `NasaApodClient.java` que realiza

la solicitud a la API y devuelve un objeto JSON con la imagen del día.

2. Modificación de la llamada a la API:

- Modifica la construcción de la URL en el método `getApodData()` para que incluya una fecha aleatoria generada en la misma clase.
- Implementa la función `getRandomDate()` que genere una fecha aleatoria dentro de los últimos 10 años en formato YYYY-MM-DD y utilízala en la construcción de la URL.

Bonus 1: Mostrar información en TextViews

1. Modificar la Activity:

- Modifica tu Activity para que, además de mostrar la imagen, extraiga y muestre el título, la fecha y la explicación de la imagen en TextViews.

2. Añadir TextViews al layout:

- Asegúrate de tener TextViews en tu layout (`activity_main.xml`) para mostrar el título, la fecha y la explicación de la imagen.

3. Actualizar la UI:

- En tu Activity, actualiza los TextViews con los datos obtenidos de la respuesta JSON.

Ojo previamente debes crear los componentes en el layout!!!

```
// Extraer la información de la respuesta JSON val imageUrl =  
apodData.getString("url") val title = apodData.getString("title") val date =  
apodData.getString("date") val explanation = apodData.getString("explanation")
```

Bonus 2: Implementar otro método de la API de la NASA (Para Notaza)

1. Explorar la documentación de la API de la NASA:

- Visita <https://api.nasa.gov/> para revisar otros endpoints disponibles, como:
 - **Mars Rover Photos:** Fotos capturadas por los rovers en Marte.
 - **Asteroids – NeoWs:** Información sobre objetos cercanos a la Tierra.

2. Implementar otro método:

- Elige otro método de la API, crea una nueva función en tu clase `NasaApodClient` para llamar a este método y obtener los datos.
- Muestra la información obtenida en tu Activity mediante un nuevo TextView o componente de UI.

Clase base proporcionada

```
package com.eusa.miapinasa;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.net.HttpURLConnection;
import java.net.URL;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Random;
import org.json.JSONException;
import org.json.JSONObject;

// Clase que se encarga de realizar la solicitud a la API de la NASA para obtener los datos
// de la foto del día
public class NasaApodClient {

    // URL base de la API de la NASA para obtener la foto del día
    private static final String NASA_APOD_API_URL =
"https://api.nasa.gov/planetary/apod";
    // Clave de la API (sustitúyela con tu propia clave obtenida)
    private static final String API_KEY = "YOUR_API_KEY";

    // Método público que realiza la solicitud a la API y devuelve un objeto JSON con los
    // datos de la respuesta
    public JSONObject getApodData() throws IOException, JSONException {
        // Generar una fecha aleatoria y agregarla a la URL
        String randomDate = getRandomDate();
        String urlWithParams = NASA_APOD_API_URL + "?api_key=" + API_KEY + //Añadir
el date!!!!
        // Se crea un objeto URL con la URL construida
        URL url = new URL(urlWithParams);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");

        int responseCode = conn.getResponseCode();
        if (responseCode != HttpURLConnection.HTTP_OK) {
            throw new IOException("Error en la solicitud: " + responseCode);
        }

        BufferedReader in = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
        String inputLine;
        StringBuilder response = new StringBuilder();

        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();
    }
}
```

```
// Se convierte la respuesta en un objeto JSON y se retorna
return new JSONObject(response.toString());
}

// Método para generar una fecha aleatoria en el pasado (por ejemplo, en los últimos 10
años)
private String getRandomDate() {
    //Desarrollar metodo

}
}
```